DAY 30

Final Project Presentation, Viva Preparation, Lessons Learned, and Course Consolidation

Software Engineering & Project Management • Student Learning Book

Learning outcomes

• Present a software project clearly, demonstrating its purpose, design, implementation, and evidence of quality.

• Answer viva questions by explaining concepts and applying them to project decisions.

• Reflect on project outcomes, teamwork, risks, defects, and lessons learned.

• Summarize the end-to-end software engineering lifecycle from requirements to maintenance.

• Prepare a project handover and identify realistic next steps.

Final case study: The College Event Management System (CEMS) lets students browse published events and register when seats are available. Organizers publish events and manage capacity. Use the project to connect the full 30-session course.

1. The Final Presentation: Tell the Project Story

A project presentation is not a list of technologies or screenshots. It is a concise explanation of a problem, the people affected, the solution designed, the work completed, the evidence collected, and the lessons learned. The audience should understand what value the product provides and what remains incomplete.

Section	What to explain	Evidence or visual
Problem and users	What is difficult today, and who experiences it?	Problem statement, stakeholder list
Objectives and scope	What did the team intend to deliver? What was excluded?	Scope statement and success criteria
Requirements	What are the key functional and non-functional needs?	User stories, SRS, acceptance criteria
Design	How is the solution structured and why?	Architecture diagram, UML, data model
Implementation	What important features were built?	Live product, code walkthrough, repository
Testing and quality	How was behaviour checked and what were the results?	Test cases, BDD scenarios, defect log
Project management	How was work planned and tracked?	WBS, schedule, Jira board, risk register
Outcome and next steps	What was achieved, what limitations remain, and what comes next?	Results, known issues, roadmap

2. Suggested 8–10 Minute Presentation Structure

Time	Activity	Presentation guidance
0:00–1:00	Problem and objectives	Explain the real need before naming technologies
1:00–2:00	Users and requirements	Show the most important user stories and constraints
2:00–3:00	Architecture and design	Explain major components and one important design decision
3:00–5:00	Product demonstration	Show one end-to-end user journey using prepared data
5:00–6:00	Testing and quality	Present selected tests, outcomes, and one defect addressed
6:00–7:00	Planning and collaboration	Show work breakdown, board, version control, or risk response
7:00–8:00	Results and limitations	State what is complete, what is not, and why

Time	Activity	Presentation guidance
8:00–10:00	Questions	Answer clearly and refer to evidence; avoid guessing

If the time limit is shorter, reduce the number of slides rather than speaking faster. Rehearse the complete presentation, including transitions and the demonstration.

3. Demonstration Checklist

• Use a stable build and prepare synthetic demo accounts and data.

• Test the exact demo sequence in the same environment before presenting.

• Choose a meaningful user journey rather than clicking through every screen.

• Show a normal path and one important boundary or error case.

• Keep a backup plan, such as screenshots or a short recording, if the environment fails.

• Be transparent about mocked features, unfinished work, and known defects.

• Ensure all team members can explain the project and their contributions.

CEMS demo flow: open the published event list → inspect event details → register as a student → show the confirmation reference → attempt a duplicate or full-event registration → explain the expected protection and the test evidence.

4. Viva Preparation: Explain, Apply, Justify

A viva evaluates understanding, not only memorized definitions. A strong answer defines the concept, connects it to the project, explains why the team chose an approach, and acknowledges limitations when appropriate.

Four-part answer pattern: (1) define the concept, (2) explain how it works, (3) apply it to your project, and (4) justify the choice or mention a limitation.

Example viva questions

Why did you choose a particular SDLC model?

Answer guide: Explain how requirements uncertainty, delivery constraints, feedback needs, risk, and team capability influenced the choice. Avoid saying one model is always best.

What is the difference between verification and validation?

Answer guide: Verification checks whether work conforms to specified requirements; validation checks whether the resulting product meets user needs and intended use.

Why is an SRS important?

Answer guide: It records agreed requirements and constraints and provides a basis for design, estimation, validation, and testing.

Explain cohesion and coupling.

Answer guide: Cohesion describes how closely related the responsibilities inside a module are; coupling describes dependencies between modules. High cohesion and low coupling are generally desirable.

What is the difference between black-box and white-box testing?

Answer guide: Black-box testing checks behavior against inputs and outputs without relying on internal code structure; white-box testing uses knowledge of internal logic and paths.

Why are both functional and non-functional requirements needed?

Answer guide: Functional requirements describe what the system does; non-functional requirements describe quality attributes or constraints such as performance, usability, security, and availability.

How did you estimate project effort?

Answer guide: Describe the estimation method and assumptions used, the size or complexity considered, and how uncertainty was handled.

What is the critical path?

Answer guide: The longest-duration path through dependent project activities; delays on it can delay the earliest project completion unless the schedule is changed.

What is a risk register?

Answer guide: A record of risks, their likelihood and impact, responses, owners, and review status.

Why use Git?

Answer guide: To record changes, support collaboration and review, and help trace or recover versions.

What is the difference between an Epic, Story, Task, and Bug?

Answer guide: An Epic groups a larger body of work; a Story describes a user-oriented outcome; a Task tracks a unit of work; a Bug records behavior that differs from expectation.

What does a Scrum Retrospective achieve?

Answer guide: The team reflects on how it worked and agrees on improvements; it is distinct from the Sprint Review, which focuses on the product and stakeholder feedback.

What is BDD?

Answer guide: A collaborative approach that clarifies desired behavior using concrete examples, often expressed in Given–When–Then form.

How do you know a feature is done?

Answer guide: It meets acceptance criteria and the agreed Definition of Done, with suitable review and test evidence.

What would you improve if you had another week?

Answer guide: Name a specific improvement, explain its user or quality value, and describe how you would verify the result.

5. How to Answer Difficult Questions

• **If you do not know:** state what you do know, identify the uncertain part, and explain how you would verify it. Do not invent an answer.

• **If a feature is incomplete:** acknowledge it, explain the reason and impact, and state the next step.

• **If asked why an alternative was not chosen:** compare options against project constraints and trade-offs.

• **If a defect is demonstrated:** explain the expected behavior, known cause if established, impact, and corrective action.

• **If asked about individual contribution:** distinguish your work from the team's work and give concrete examples.

6. Lessons Learned and Retrospective

Lessons learned should be specific and actionable. “Communication should improve” is too vague unless the team identifies what communication failed and what it will change.

Reflection area	Useful question	Example improvement
Requirements	Which assumption changed after feedback?	Review acceptance criteria with a stakeholder before implementation
Estimation	Which task took longer than expected and why?	Split uncertain tasks and run a small technical investigation first
Quality	Which defect escaped early testing?	Add a regression test for the failed behavior
Collaboration	Where did work become blocked?	Make dependencies visible and discuss blockers earlier
Tools and workflow	Which status or handoff caused confusion?	Define clear entry/exit criteria for board statuses
Documentation	What was hard for another person to understand?	Add setup steps and record important design decisions

A retrospective should result in a small number of actions that have owners and can be checked later. The goal is learning and improvement, not blaming individuals.

7. Course Consolidation: Lifecycle at a Glance

Stage	Main question	Typical outputs
Requirements engineering	What do users and stakeholders need?	Requirements, SRS, acceptance criteria

Stage	Main question	Typical outputs
Design	How should the solution be structured?	Architecture, UML, data model, interfaces
Planning and estimation	What work is needed, in what order, and with what risks?	WBS, schedule, estimates, risk register
Implementation and configuration management	How will changes be created and controlled?	Code, Git history, reviews, builds
Testing and quality assurance	Does the product meet requirements and quality expectations?	Test plan, test cases, results, defect reports
Delivery and maintenance	Can the product be released, supported, and improved?	Release notes, handover, maintenance backlog

These stages are connected and may overlap. Agile teams revisit requirements, design, testing, and planning throughout development instead of treating them as one-time steps.

8. Final Project Submission Checklist

• Project overview with problem, objectives, scope, stakeholders, and constraints.

• SRS or requirements list with functional and non-functional requirements.

• Design artefacts appropriate to the project, such as architecture and UML diagrams.

• Implementation repository with meaningful commits and setup instructions.

• Jira or equivalent backlog showing work items, priorities, and status.

• Test cases and results, including boundary or failure cases.

• Defect log and evidence of retesting important fixes.

• Risk register and explanation of key project decisions.

• User guide, release notes, known limitations, and handover information.

• Presentation slides and a rehearsed demonstration.

9. Suggested Final Evaluation Rubric

Criterion	Marks	Evidence expected
Problem definition and requirements	15	Clear scope, stakeholders, requirements, acceptance criteria
Design and technical reasoning	15	Suitable design artefacts and justified decisions
Implementation and version control	15	Working features, organized repository, traceable changes
Testing and quality	20	Relevant test coverage, results, defect handling
Project planning and tools	10	Backlog, schedule or plan, risks, progress tracking
Demonstration and presentation	10	Clear explanation and credible end-to-end demonstration
Viva and reflection	15	Conceptual understanding, justification, lessons learned

This is a sample 100-mark rubric. Instructors should adapt it to institutional assessment rules and communicate the final criteria before submission.

10. Final Self-Assessment

Mark each statement honestly before submitting your project.

■ I can explain the problem and the users this project serves.

■ I can trace key requirements to design decisions and tests.

■ I can explain the chosen SDLC or Agile approach and its trade-offs.

■ I can show how Git and the project board were used.

■ I can explain at least one important defect and how it was verified after a fix.

■ I can discuss project risks, limitations, and future improvements.

■ I can demonstrate the main user journey without relying on unverified claims.

■ I can distinguish my own contribution from the team's contribution.

11. Closing Review Questions

What is the most important purpose of software engineering?

Suggested answer: To apply systematic, disciplined approaches to building, operating, and maintaining software that meets real needs.

Why does a project need both design and testing?

Suggested answer: Design structures the solution; testing provides evidence that the implemented behavior meets expectations.

Why is project management needed in technical work?

Suggested answer: To coordinate scope, time, effort, people, dependencies, risks, and communication.

What makes a project presentation credible?

Suggested answer: Clear reasoning, honest scope, working evidence, known limitations, and the ability to answer questions.

What should a team do after a project is delivered?

Suggested answer: Hand over relevant knowledge, monitor issues, maintain the product, and use lessons learned to improve future work.

12. Final Homework / Reflection

1. Write a one-page reflection on three lessons learned from your project, with one concrete improvement for each lesson.

2. Prepare a five-question viva practice sheet and answer each question using an example from your project.

3. Complete the final submission checklist and identify any missing evidence honestly.

4. Write a short handover note for the next team that will maintain or extend the project.

Course completion: You have connected the main activities of software engineering and project management—from identifying needs and designing a solution to planning work, controlling changes, testing quality, presenting evidence, and preparing for maintenance.